

week 5

Day 1

- no classes

Day 2

- Continued on working on our codes
- Worked on the code from last week
- Tried out different usages of the morse code

```
// {"P5LIVE":{"name":"Morsecode 3","mod":1780319932110}}

let bgColor;
let typedText = "";
let morseItems = [];

let synth;

// Größe der Morsezeichen
const morseSize = 2;

// Morse Alphabet
const morse = {
  a: ".-", b: "-...", c: "-.-.", d: "-..",
  e: ".", f: "..-", g: "--.", h: "....",
  i: "..", j: ".---", k: "-.-", l: "-... ",
  m: "--", n: "-.", o: "---", p: ".--.",
  q: "--.-", r: ".-.", s: "...", t: "-",
  u: "..-", v: "...-", w: ".--",
  x: "-.-.", y: "-.-", z: "--.."
};

function setup() {
  createCanvas(windowWidth, windowHeight);
  bgColor = color(255, 0, 127);

  // Sound
  synth = new p5.Oscillator('sine');
  synth.start();
  synth.amp(0);

  textFont('monospace');
}
```

```

function draw() {
  background(bgColor);

  // Morsezeichen zeichnen
  for (let item of morseItems) {

    // leichte Eigenbewegung
    item.targetX += random(-0.3, 0.3);
    item.targetY += random(-0.3, 0.3);

    // Abstand zu anderen prüfen
    for (let other of morseItems) {
      if (item !== other) {
        let d = dist(item.x, item.y, other.x, other.y);

        // Mindestabstand
        if (d < 170) {
          let angle = atan2(
            item.y - other.y,
            item.x - other.x
          );
          let force = (170 - d) * 0.05;
          item.targetX += cos(angle) * force;
          item.targetY += sin(angle) * force;
        }
      }
    }

    // im Bildschirm halten
    item.targetX = constrain(item.targetX, 100, width - 100);
    item.targetY = constrain(item.targetY, 100, height - 200);

    // weiche Bewegung
    item.x = lerp(item.x, item.targetX, 0.03);
    item.y = lerp(item.y, item.targetY, 0.03);

    drawMorse(item);
  }

  // Getippter Text unten
  fill(255);
  textAlign(CENTER);
  textSize(40);
  text(typedText, width / 2, height - 80);
}

// Tastatureingabe
function keyTyped() {

  let k = key.toLowerCase();

```

```

// Buchstaben
if (morse[k]) {
  typedText += k;
  bgColor = color(random(255), random(255), random(255));
let item = {
  code: morse[k],

  // startet in der Mitte
  x: width / 2,
  y: height / 2,

  // Zielposition
  targetX: random(150, width - 150),
  targetY: random(150, height - 150)
};

morseItems.push(item);

// alle bestehenden Zeichen verschieben
for (let other of morseItems) {
  other.targetX += random(-100, 100);
  other.targetY += random(-100, 100);

  // innerhalb Bildschirm halten
  other.targetX = constrain(other.targetX, 100, width - 100);
  other.targetY = constrain(other.targetY, 100, height - 100);
}

  playMorse(morse[k]);
}

// Leerzeichen = nur Pause
if (k === " ") {
  typedText += " ";
  // kleine stille Pause
  setTimeout(() => {}, 500);
}
}

// Backspace
function keyPressed() {
  if (keyCode === BACKSPACE) {
    typedText = typedText.slice(0, -1);
    // nur löschen wenn letztes Zeichen Morse war
    if (morseItems.length > 0) {
      morseItems.pop();
    }
  }
}

```

```

// Morse zeichnen
function drawMorse(item) {
  let spacing = 30 * morseSize;
  for (let i = 0; i < item.code.length; i++) {

    let sym = item.code[i];

    let px = item.x + (i - item.code.length / 2) * spacing;
    let py = item.y;

    fill(255);
    noStroke();
    textAlign(CENTER, CENTER);
    textSize(40 * morseSize);
    if (sym === ".") {
      text(".", px, py);
    }
    if (sym === "-") {
      text("-", px, py); // echter Gedankenstrich (en dash)
    }
  }
}
// Morse Sound
function playMorse(code) {

  let time = 0;

  for (let sym of code) {
    let duration = sym === "." ? 120 : 350;
    setTimeout(() => {
      synth.freq(400);
      synth.amp(0.35, 0.02);
      setTimeout(() => {
        synth.amp(0, 0.05);
      }, duration);
    }, time);
    time += duration + 120;
  }
}

```

[P5L_Morsecode 3_20260601131852.png](#)

```

// {"P5LIVE":{"name":"Morsecode to text","mod":1780320255832}}

let morseInput = "";

// Morse Alphabet
let morse = {
  ".-": "A",

```

```

    "-...": "B",
    "-.-.": "C",
    "-..": "D",
    ".": "E",
    "..-": "F",
    "--.": "G",
    "....": "H",
    "..": "I",
    ".---": "J",
    "-.-": "K",
    "-...": "L",
    "--": "M",
    "-.": "N",
    "---": "O",
    ".--.": "P",
    "--.-": "Q",
    "-.-.": "R",
    "...": "S",
    "-": "T",
    "..-": "U",
    "...-": "V",
    ".--": "W",
    "-...-": "X",
    "-.---": "Y",
    "--..": "Z",
};

function setup() {
  createCanvas(windowWidth, windowHeight);
  textSize(70);
  textAlign(CENTER, CENTER);
}

function draw() {

  background(0);
  fill(255);

  // oben: Morsecode
  text(morseInput, width / 2, height / 3);

  // unten: Übersetzung
  let translated = translateMorse(morseInput);

  text(translated, width / 2, height / 1.5);
}

// Morse → Text
function translateMorse(str) {

```

```

// Wörter mit 3 Leerzeichen trennen
let words = str.split("   ");

let result = "";

for (let w = 0; w < words.length; w++) {

    // einzelne Buchstaben trennen
    let letters = words[w].split(" ");

    for (let i = 0; i < letters.length; i++) {

        if (morse[letters[i]]) {
            result += morse[letters[i]];
        }
    }

    // Leerzeichen zwischen Wörtern
    result += " ";
}

return result;
}

// Tastatur
function keyPressed() {

    // löschen
    if (keyCode === BACKSPACE) {
        morseInput = morseInput.slice(0, -1);
    }

    // erlaubte Zeichen
    if (key === "." || key === "-" || key === " ") {
        morseInput += key;
    }

    return false;
}

'// noprotect'

```

[P5L_Morsecode to text_20260601132415.png](#)

```

// {"P5LIVE":{"name":"morsecode x textanimation","mod":1780320369697}}

let colorText;
let colorBG;

let currentWord = "";

```

```

let words = [];
let morseWords = [];

// Morse-Tabelle
let morseMap = {
  a: ".-", b: "-...", c: "-.-.", d: "-..", e: ".",
  f: "..-.", g: "--.", h: "....", i: "..", j: ".---",
  k: "-.-", l: ".-..", m: "--", n: "-.", o: "---",
  p: ".--.", q: "--.-", r: ".-.", s: "...", t: "-",
  u: "..-", v: "...-", w: ".--", x: "-..-", y: "-.--",
  z: "--.."
};

function setup() {
  createCanvas(windowWidth, windowHeight);
  colorText = color(255);
  colorBG = color(0, 0, 255);
  frameRate(10);
}

function wordToMorse(word) {
  word = word.toLowerCase();
  let result = "";

  for (let i = 0; i < word.length; i++) {
    let char = word[i];
    if (morseMap[char]) {
      result += morseMap[char] + " ";
    }
  }

  return result.trim();
}

function draw() {

  // Farbsteuerung
  if (mouseIsPressed) {
    if (mouseX < width / 2) {
      colorText = color(random(255), random(255), random(255));
    } else {
      colorBG = color(random(255), random(255), random(255));
    }
  }

  background(colorBG);
  fill(colorText);

  textFont('monospace');

```

```

// =====
// OBEN: INPUT ANZEIGE
// =====
textSize(40);
textAlign(LEFT);

let displayInput = words.join(" ") + (currentWord.length > 0 ? " " +
currentWord : "");

text(displayInput, 100, 60);

// =====
// MORSE ANIMATION
// =====
textSize(100);
textWrap(CHAR);
textAlign(LEFT);
textStyle(random([NORMAL, ITALIC]));

let live = frameCount % 10;
let sine = floor(5 * sin(frameCount / 10) + 5);

textLeading(30 * (live / 2));

if (morseWords.length > 0) {
    let index = sine % morseWords.length;
    text(morseWords[index].repeat(50), 100, 150, windowWidth/1.1,
windowHeight);
}
}

// Tastatur → Wort bauen
function keyTyped() {
    if (key === " ") {
        if (currentWord.length > 0) {
            words.push(currentWord);
            morseWords.push(wordToMorse(currentWord));
            currentWord = "";
        }
    } else {
        currentWord += key;
    }
}

// Steuerung
function keyPressed() {

    if (keyCode === ENTER) {
        if (currentWord.length > 0) {
            words.push(currentWord);

```



```

        morseWords.push(wordToMorse(currentWord));
        currentWord = "";
    }
}

if (keyCode === BACKSPACE) {
    currentWord = currentWord.slice(0, -1);
    return false;
}

if (keyCode === ESCAPE) {
    words = [];
    morseWords = [];
    currentWord = "";
}
}

```

[P5L_morsecode x textanimation_20260601132609.png](#)

```

// {"P5LIVE":{"name":"Morse Flash","mod":1780320497107}}

//Text speichern, leer starten
let inputText = "";
let morseQueue = [];
let isPlaying = false;

let morseMap = {
  a: ".-",
  b: "-...",
  c: "-.-.",
  d: "-..",
  e: ".",
  f: "..-.",
  g: "--.",
  h: "....",
  i: "..",
  j: ".---",
  k: "-.-",
  l: ".-..",
  m: "--",
  n: "-.",
  o: "---",
  p: ".--.",
  q: "--.-",
  r: "-.-",
  s: "...",
  t: "-",
  u: "..-",
  v: "...-",

```

```

    w: ".--",
    x: "-...-",
    y: "-.---",
    z: "--..".
};

let bgColor = 0;

function setup() {
    createCanvas(windowWidth, windowHeight);
    textAlign(CENTER, CENTER);
    textSize(40);
}

function draw() {
    //wert von bgColor übernehmen
    background(bgColor);
    fill(255);
    // Text weiss und zentriert
    text(inputText, width / 2, height / 2);
}

function startMorse() {
    //Input von inputText in morsezeichen übersetzen und in morseString
    speichern
    let morseString = textToMorse(inputText);
    //einzelne Morsezeichen aufteilen und in morseQue speichern
    morseQueue = morseString.split("");
    isPlaying = true;
    //playNext Funktion abrufen
    playNext();
}

function playNext() {
    //wenn Queue leer = alle Symbole wurden abgespielt
    if(morseQueue.length === 0) {
        //Hintergrund wieder schwarz
        bgColor = 0;
        isPlaying = false;
        return;
    }
    // Symbol aus der Queue holen und entfernen und in symbol speichern
    let symbol = morseQueue.shift();
    //wenn symbol . dann kurzer flash
    if(symbol === ".") {
        flash(120);
    }
    //wenn symbol - dann langer flash

```

```

    else if(symbol === "-") {
        flash(350);
    }
    //wenn symbol Leerzeichen, warten
    else {
        setTimeout(playNext, 180);
    }
}

```

```

function flash(duration) {
    //Hintergrund Rot
    bgColor = color(5, 20, 150);
    // nach duration ms Hintergrund wieder schwarz
    setTimeout(() => {
        bgColor = 0;
        //nach 100ms Pause nächstes Symbol
        setTimeout(playNext, 100);
    }, duration);
}

```

```

function textToMorse(text) {
    text = text.toLowerCase();

    let result = "";

    for(let i = 0; i < text.length; i++) {
        let c = text[i];

        if(c === " ") {
            result += "  ";
        }
        else if(morseMap[c]) {
            result += morseMap[c] + " ";
        }
    }

    return result.trim();
}

```

```

// =====
// INPUT
// =====
function keyPressed() {

```

```

    if(keyCode === ENTER) {
        if(!isPlaying) {
            startMorse();
        }
    }
}

```

```

    }

    if(keyCode === BACKSPACE) {
        inputText = inputText.slice(0, -1);
    }

    if(key.length === 1 && !isPlaying) {
        inputText += key;
    }

    return false;
}

```

[P5L_Morse Flash_20260601132817.png](#)

Day 3

- We tried to turn our previous codes into a concept
- We ran into some difficulties; we couldn't find a common thread to tie it all together.
- We just couldn't seem to make any progress.
- Then we discovered that we both played the piano
- That inspired us, and we came up with the idea of using music as a language instead of Morse code
- We started with a song we'd both played before and used the sheet music as our starting point
- We split up the work: Samir further developed the audio using Strudel, and I worked on the visuals
- For the visuals, I took a closer look at the visual elements of the "language of music"
- I reduced the individual elements to their basic components and tried to create visuals using circles, lines, and rectangles

```

// {"P5LIVE":{"name":"musik 1","mod":1780321724801}}

let margin = 30;
let gap = 20;

// Verschiedene Startphasen damit alle versetzt schwingen
let phase = [0.0, 1.2, 2.4, 3.6, 4.8];
let speed = [0.7, 1.1, 0.5, 2, 1.3];

function setup() {
    createCanvas(windowWidth, windowHeight);
    noStroke();
}

function draw() {

```

```

background(20, 0, 255);
fill(235);

let h = height - margin * 2 - gap;
let colW = width / 5;
let w = colW * 0.75;

for (let i = 0; i < 5; i++) {

  // sin() geht von -1 bis +1 → +1 verschiebt auf 0 bis 2 → /2 = 0 bis 1
  let ratio = map(sin(frameCount * 0.01 * speed[i] + phase[i]), -1, 1, 50/h,
  1 - 50/h);

  let cx = colW * (i + 0.5);

  if (i % 2 == 0) {
    let r = h * ratio;
    let e = h - r;
    rect(cx - w/2, margin, w, r);
    ellipse(cx, margin + r + gap + e/2, w, e);
  } else {
    let e = h * ratio;
    let r = h - e;
    ellipse(cx, margin + e/2, w, e);
    rect(cx - w/2, margin + e + gap, w, r);
  }
}

function windowResized() {
  resizeCanvas(windowWidth, windowHeight);
}

```

[P5L_music 1_20260601134844.png](#)

```

// {"P5LIVE":{"name":"music 2","mod":1780321738887}}

function setup() {
  createCanvas(windowWidth, windowHeight);
}

function draw() {
  background(0, 0, 255, 10);
  stroke(255);
  strokeWeight(3);

  let spacing = height / 6;
  let t = frameCount / 60;

```

```

for (let k = 0; k < 3; k++) {
  let phase = (k / 3) * TWO_PI; // die 3 Trichter versetzt in der Zeit

  for (let i = 1; i <= 5; i++) {
    // links: eng zusammen, rechts: weit auseinander – oder umgekehrt
    let leftSpacing = map(sin(t + phase), -1, 1, 20, spacing);
    let rightSpacing = map(sin(t + phase), -1, 1, spacing, 20);

    let leftY = height / 2 + (i - 3) * leftSpacing;
    let rightY = height / 2 + (i - 3) * rightSpacing;

    line(0, leftY, width, rightY);
  }
}

```

[P5L_music 2_20260601134858.png](#)

```

// {"P5LIVE":{"name":"musik 3","mod":1780321754047}}

function setup() {
  createCanvas(windowWidth, windowHeight)

  setupAudio(true) // global vars
  // a5.ease = .075 // set easing
}

function draw() {
  /* audio vars: amp, ampEase, fft, fftEase, waveform, waveformEase */
  updateAudio()

  let size = 250
  let blink = 60
  background(255)
  noStroke()

  k = frameCount%blink

  fill(0, 0, 255)
  if (k > blink/2) {
    fill(255)
    background(0,0,255)
  }
  rect(0, 0, width / 2, height / 2)
  rect(width / 2, height / 2, width / 2, height / 2)

```

```

    fill(255)

    if (k > blink/2) {
        fill(0,0,255)
    }
    ellipse(width / 4, height / 4, size * fftEase[80] * 0.01 )
    ellipse(3 / 4 * width, 3 / 4 * height, size * fftEase[100] * 0.01)

    fill(0, 0, 255)
    if (k > blink/2) {
        fill(255)
    }
    ellipse(3 / 4 * width, height / 4, size * fftEase[50] * 0.01)
    ellipse(1 / 4 * width, 3 / 4 * height, size * fftEase[0] * 0.01)

}

/*
P5LIVE - Audio
If using outside P5LIVE, include p5live-audio.js
https://cdn.jsdelivr.net/gh/ffd8/P5LIVE/includes/utils/p5live-audio.js
*/

```

[P5L_music 3_20260601134914.png](#)

```

// {"P5LIVE":{"name":"music 4","mod":1780321773649}}

/*
_HY5_p5_hydra // cc teddavis.org 2024
pass p5 into hydra
docs: https://github.com/ffd8/hy5
*/

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js', 'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']

// // sandbox - start
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra
P5.toggle(0) // optionally hide p5

src(s0)
.kaleid([3,8,1,4])
.scrollX(1,0.5)
.out()

```

```
// sandbox - end

function setup() {
  createCanvas(windowWidth, windowHeight);
  setupAudio(true) // global vars
  // a5.ease = .075 // set easing
}

function draw() {
  /* audio vars: amp, ampEase, fft, fftEase, waveform, waveformEase */
  updateAudio()
  background(0, 0, 255, 10);
  noFill();
  let speed = 20;
  let t = frameCount / speed;

  // Weisse Linien
  stroke(255);
  strokeWeight(3);

  // line(0, baseY, width, baseY); // gerade Linien

  for (let i = 1; i <= 5; i++) {
    let baseY = (height / 6) * i;
    beginShape();
    for (let x = 0; x <= width; x += 5) {
      let amp = sin((x / width) * PI) * 300;
      let phase = (i / 5) * TWO_PI;
      let y = baseY + sin(t + (x / width) * TWO_PI + phase) * amp;
      vertex(x, y);
    }
    endShape();
  }
}

/*
P5LIVE - Audio
If using outside P5LIVE, include p5live-audio.js
https://cdn.jsdelivr.net/gh/ffd8/P5LIVE/includes/utils/p5live-audio.js
*/
```



```

// {"P5LIVE":{"name":"music 5","mod":1780321810433}}

/*
  _HY5_p5_hydra // cc teddavis.org 2024
  pass p5 into hydra
  docs: https://github.com/ffd8/hy5
*/

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-
synth.js', 'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js',
'includes/libs/hy5.js']

// // sandbox - start
// H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra
P5.toggle(0) // optionally hide p5

src(s0)
.kaleid([3,8,2,4])
.scrollX(1,0.5)
.out()
// sandbox - end


function setup() {
  createCanvas(windowWidth, windowHeight);
  setupAudio(true) // global vars
  // a5.ease = .075 // set easing
}

function draw() {
  updateAudio()
  background(255, 255, 255,10);
  strokeWeight(2);
  let spacing = height / 6;
  let speed = 2 * fftEase[80];

  for (let i = 1; i <= 5; i++) {
    if (i == 1 || i == 5) {
      stroke(255, 0, 150); // <-- pink
    } else {
      stroke(0, 0, 255);
    }
    let intens = 100;
    let offset = sin(frameCount / speed + i) * intens;
    line(0, i * spacing + offset, width, i * spacing + offset);
  }
}

```

```

    }
  }

  /*
  P5LIVE - Audio
  If using outside P5LIVE, include p5live-audio.js
  https://cdn.jsdelivr.net/gh/ffd8/P5LIVE/includes/utils/p5live-audio.js
  */

```

[P5L_music_5_20260601135010.png](#)

Day 4 and 5

- Worked on my code for the final presentation

Here my final code:

```

// {"P5LIVE":{"name":"01","mod":1780327672806}}

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js', 'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']

// sandbox - start
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra
P5.toggle(0) // optionally hide p5

src(s0)
//scrolling effect
.scrollY(1,-.5)
  .out()
// sandbox - end

function setup() {
  createCanvas(windowWidth, windowHeight);
}

function drawNote(x, y, size, highlight) {
  if (highlight) {
    fill(255);
    rect(x - size * 0.8, y - size * 3.2, size * 1.8, size * 4.5);
    fill(0, 0, 255);
  } else {
    fill(255);
  }
}

```

```

    ellipse(x, y, size * 1.4, size);
    rect(x + size * 0.5, y - size * 3, size * 0.2, size * 3);
}

function draw() {
  background(0, 0, 255);
  noStroke();

  //Play the steps one after another
  let bpm = 130;
  let duration = 60 / bpm * 60;
  let step = floor(frameCount / duration) % 4;

  //let step = 3

  //Define the individual steps here
  let size, cols, rows;
  if (step == 0 ){size = 200, cols = 3, rows = 1;}
  if (step == 1 ){size = 100, cols = 6, rows = 2;}
  if (step == 2 ){size = 50, cols = 12, rows = 4;}
  if (step == 3 ){size = 20, cols = 20, rows = 10;}

  // Frames per note and position 0 to cols for highlight
  let highlightSpeed = 5;
  let pos = floor(frameCount / highlightSpeed) % cols;

  for (let row = 0; row < rows; row++) {
    let highlightCol;
    if (row % 2 == 0) {
      highlightCol = (pos + row * 3) % cols;
    } else {
      highlightCol = cols - 1 - (pos + row * 3) % cols;
    }

    for (let col = 0; col < cols; col++) {
      let x = (col + 0.5) * (width / cols);
      let y = (row + 0.5) * (height / rows) + size;
      drawNote(x, y, size,col == highlightCol);
    }
  }
}

```

[P5L_01_20260601152752.png](#)
[01.mov](#)

```
// {"P5LIVE":{"name":"02","mod":1780327988525}}
```

```

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js', 'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']

// sandbox - start
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra
P5.toggle(0) // optionally hide p5

src(s0)
//scroll effect
    .scrollX(1, .5)

    .out()
// sandbox - end

let bpm = 130
let margin = 30;
let gap = 20;

// Different start phases so that everyone swings in a staggered pattern
let phase = [0.0, 1.2, 2.4, 3.6, 4.8];

function setup() {
    createCanvas(windowWidth, windowHeight);
    noStroke();
    setupAudio(true) // global vars
    // a5.ease = .075 // set easing
}

function draw() {
    /* audio vars: amp, ampEase, fft, fftEase, waveform, waveformEase */
    updateAudio()
    //speed audio reactive by using fftEase
    let speed = [fftEase[30]*0.0002, fftEase[100]*0.0007, fftEase[80]*0.0002, fftEase[120]*0.0007, fftEase[5]*0.0002];

    let invertDuration = 60 / bpm * 60 * 4; // * 4 = change every 4th beat
    let invert = floor(frameCount / invertDuration) % 2;

    // Alternating background colors
    if(invert){
        background(255)
    }else{
        background(color(0,0,255))
    }

    // Alternating fill colors

```

```

if(invert){
  fill(color(0,0,255))
}else{
  fill(255)
}

let h = height - margin * 2 - gap;
let colW = width / 5;
let w = colW * 0.75;

for (let i = 0; i < 5; i++) {

  // sin() ranges from -1 to +1 → shifting to +1 results in a range of 0 to
  2 → /2 = 0 to 1
  let ratio = map(sin(frameCount * 0.01 * speed[i] + phase[i]), -1, 1, 50/h,
  1 - 50/h);

  let cx = colW * (i + 0.5);

  if (i % 2 == 0) {
    let r = h * ratio;
    let e = h - r;
    rect(cx - w/2, margin, w, r);
    ellipse(cx, margin + r + gap + e/2, w, e);
  } else {
    let e = h * ratio;
    let r = h - e;
    ellipse(cx, margin + e/2, w, e);
    rect(cx - w/2, margin + e + gap, w, r);
  }
}

}

/*
P5LIVE - Audio
If using outside P5LIVE, include p5live-audio.js
https://cdn.jsdelivr.net/gh/ffd8/P5LIVE/includes/utls/p5live-audio.js
*/

```

[P5L_02_20260601153308.png](#)

[02.mov](#)

```

// {"P5LIVE":{"name":"03","mod":1780328118126}}

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js', 'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']

// sandbox - start
// H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra
P5.toggle(0) // optionally hide p5

src(s0)
  //scroll effect
  .scrollX(1, -.5)

  .out()
// sandbox - end

function setup() {
  createCanvas(windowWidth, windowHeight)
  setupAudio(true)
}

function draw() {
  updateAudio()

  let size = 250
  //Color change speed (lower value = faster)
  let blink = 50
  background(255)
  noStroke()

  k = frameCount % blink

  fill(0, 0, 255)
  if(k > blink / 2) {
    fill(255)
    background(0, 0, 255)
  }
  rect(0, 0, width / 4, height / 2)
  rect(width / 2, 0, width / 4, height / 2)
  rect(width / 4, height / 2, width / 4, height / 2)
  rect(3 / 4 * width, height / 2, width / 4, height / 2)

  //Alternating circle colors
  fill(255)
  if(k > blink / 2) {

```

```

    fill(0, 0, 255)
  }
  ellipse(1 / 8 * width, 1 / 4 * height, size * fftEase[10] * 0.01)
  ellipse(5 / 8 * width, 1 / 4 * height, size * fftEase[50] * 0.01)
  ellipse(3 / 8 * width, 3 / 4 * height, size * fftEase[90] * 0.01)
  ellipse(7 / 8 * width, 3 / 4 * height, size * fftEase[110] * 0.01)

  //Alternating circle colors
  fill(0, 0, 255)
  if(k > blink / 2) {
    fill(255)
  }
  ellipse(3 / 8 * width, 1 / 4 * height, size * fftEase[30] * 0.01)
  ellipse(7 / 8 * width, 1 / 4 * height, size * fftEase[70] * 0.01)
  ellipse(5 / 8 * width, 3 / 4 * height, size * fftEase[100] * 0.01)
  ellipse(1 / 8 * width, 3 / 4 * height, size * fftEase[80] * 0.01)

}

/*
P5LIVE - Audio
If using outside P5LIVE, include p5live-audio.js
https://cdn.jsdelivr.net/gh/ffd8/P5LIVE/includes/utils/p5live-audio.js
*/

```

[P5L_03_20260601153518.png](#)

[03.mov](#)

```

// {"P5LIVE":{"name":"04","mod":1780328295923}}

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js', 'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']

// sandbox - start
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra
P5.toggle(0) // optionally hide p5

src(s0)
//scroll effect
.scrollY(1,0.5)
  .out()
// sandbox - end

let notes = [];
let words = [];

```

```

//Input words here
let wordList = ['B#5','A5','F#5','D#5','F5','E5','THANK YOU FOR YOUR
ATTENTION'];

function setup() {
  createCanvas(windowWidth, windowHeight);
  setupAudio(true);
}

//single note
function drawNote(x, y, size) {
  ellipse(x, y, size * 1.4, size);
  rect(x + size * 0.5, y - size * 3, size * 0.2, size * 3);
}

//double note
function drawDoubleNote(x, y, size) {
  let gap = size * 2;
  drawNote(x, y, size);
  drawNote(x + gap, y, size);
  rect(x + size * 0.5, y - size * 3, gap, size * 0.2);
}

//appear on keypress
function keyPressed() {
  if (key == 's') {
    notes.push({ x: random(width), y: random(height), vx: random(-3, 3),
vy: random(-3, 3), type: 'single' });
  }
  if (key == 'd') {
    notes.push({ x: random(width), y: random(height), vx: random(-3, 3),
vy: random(-3, 3), type: 'double' });
  }
  if (keyCode == ENTER) {
    words.push({
      x: random(width),
      y: random(height),
      vx: random(-3, 3),
      vy: random(-3, 3),
      word: random(wordList)
    });
  }
}

function draw() {
  updateAudio();

  let bpm = 130;
  let invertDuration = 60 / bpm * 60 * 4;
  let invert = floor(frameCount / invertDuration) % 2;

```



```

if (invert) {
  background(255);
  fill(color(0, 0, 255));
} else {
  background(color(0, 0, 255));
  fill(255);
}
noStroke();

//reaction to audio
let noteSize = map(fftEase[50], 0, 255, 15, 100);
let fontSize = map(fftEase[50], 0, 255, 20, 80);

// notes
for (let n of notes) {
  let mouseSpeed = map(mouseY, 0, height, 2, 0.1);
  n.x += n.vx * mouseSpeed;
  n.y += n.vy * mouseSpeed;
  if (n.x < 0 || n.x > width) n.vx *= -1;
  if (n.y < 0 || n.y > height) n.vy *= -1;
  if (n.type == 'single') {
    drawNote(n.x, n.y, noteSize);
  } else {
    drawDoubleNote(n.x, n.y, noteSize);
  }
}

// words
textSize(fontSize);
textAlign(CENTER, CENTER);
for (let w of words) {
  let mouseSpeed = map(mouseY, 0, height, 2, 0.1);
  w.x += w.vx * mouseSpeed;
  w.y += w.vy * mouseSpeed;
  if (w.x < 0 || w.x > width) w.vx *= -1;
  if (w.y < 0 || w.y > height) w.vy *= -1;
  text(w.word, w.x, w.y);
}

/*
P5LIVE - Audio
If using outside P5LIVE, include p5live-audio.js
https://cdn.jsdelivr.net/gh/ffd8/P5LIVE/includes/utils/p5live-audio.js
*/

```

[P5L_04_20260601153815.png](#)

[04.mov](#)